A Programming Environment for a Timeshared System

Richard P. Gabriel and Martin E. Frost

Stanford University

1. Introduction

In 1968 the Stanford Artificial Intelligence Laboratory began to construct a programming environment from a PDP-10, a pre-TOPS-10 DEC[1] timesharing system, and some innovative terminal hardware. By now this has developed into a programming environment for a KL-10 that integrates our editor with various other system functions, especially the Lisp subsystem. We use the term 'SAIL' to refer to the Stanford A. I. Lab KL-10 computer running the WAITS timesharing system. [Harvey 1982]

By 'programming environment' we mean the mechanisms that allow a user to type text at his program or subsystem, and which manage output text.[2] We are talking about mechanical management of the interaction between user and program, not about any intelligent mediation. A good programming environment should be flexible enough to suit individuals, yet without requiring the mechanics of interaction to be re-learned for each new program.

In this paper we describe our programming environment, what makes it unique, and why we think that it is not necessary to move to personal computers for a very usable programming environment.

[1] DEC, TOPS-10, and TOPS-20 are registered trademarks of Digital Equipment Corporation.

[2] Even though our computer and its hardware have graphics capabilities, we have chosen to dwell on our text handling capabilities because they are readily adapted to other systems without special graphics hardware.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

©1984 ACM 0-89791-131-8/84/0400/0185$00.75

2. Components

The components of the programming environment are:

1. a high-speed (7.5 megabaud) *video display* system with a *video switch*;
2. an *extended keyboard* with two shift-like keys called *control* and *meta*;
3. a *line editor* within the operating system for fast editing of each input line;
4. a fast display editor that exploits the display system, keyboard, and line editor; and
5. a protocol for communication between the text editor and other jobs in the system.

By combining these components we have created a uniform *user interface* to every subsystem in use on SAIL. This interface is our editor, **E**, in conjunction with the communication protocol. When **E** is used as the interface to the Lisp subsystem, we refer to the interface as the '**E**-Lisp interface.'

This paper describes these components individually, and the **E**-Lisp interface is discussed in detail.

3. The Programming Environment

There are two reasons why the SAIL system is important. First, it provides many advanced user-interface and programming-environment features—for example, display systems, large physical memories (instead of paging systems), large file systems, etc. We have had a lot of experience with display terminals, supporting them since about 1968.

Second, these programming environment features exist on a relatively old-fashioned timesharing system. There are only a few features of WAITS that are not present in virtually every timesharing system, and so we show how an existing timesharing system without advanced user-interface features can be equipped with them.

This system has influenced many other systems in significant ways.

The SAIL display system was copied by the MIT AI Lab, and that system was generalized to the bitmap displays that the Symbolics LM-2,

Symbolics 3600 [Moon 1983], and LMI CADR [Greenblatt 1983] use. The extended keyboard was also adapted by MIT and used by those companies.

E was one of the first display editors, and the development of EMACS was influenced by it.

A *video switch*, such as SAIL's, is a crossbar switch that connects any one of a number of TV monitors (for example, terminals) to any one of a number of channels of video output. Video switches are now commercially available, and LMI Lambda consoles can be interfaced to Lambda computers using a video switch.

The **E**-Lisp interface pre-dates many other editor-Lisp interfaces where the editor serves as the user interface, but in most other major similar interfaces the editor is written in Lisp. The advantage of our method is that the standard system editor—well-tailored to the hardware and the operating system—serves as the medium of interaction. When a new subsystem is run under **E**, the same editor and command structure is used. There is nothing new for the user to learn.

4. System Support

A few key hardware and operating system support features add to the effectiveness of the programming environment; the **E** editor and the user interface are built upon these features. They are the *Data Disc display system*, the *video switch*, the *line editor*, and the *extended keyboard.*

The most important special facility at SAIL is the Data Disc display system, which holds 32 video images on its (now semiconductor) 'disk.' The Data Disc has a DMA interface which allows the timesharing system to put out a full screen display in 1/30th of a second. Each of the 32 video outputs (called a 'channel') from the Data Disc can be routed through a video switch (under software control) to any Data Disc terminal, which consists of a standard television monitor and an independent keyboard. Such a Data Disc terminal provides the user with a high-speed display.

The effect of the Data Disc is that moving around in a file with **E** is fast. The user is never waiting for output of displayed text to finish, because it is virtually instantaneous.

Moreover, with the video switch the user can have several jobs each appearing on a different channel of the Data Disc, but with all such virtual terminals controlled from the same physical terminal. This is useful, for example, for viewing program text with one job running **E** while debugging the program with another job running Lisp. Here one physical terminal is connected to many virtual terminals, with switching among them instantaneous thanks to the video switch. The opposite situation, in which many physical terminals are connected to the same virtual terminal, is similarly convenient. The video switch allows people to view the same virtual terminal (i.e., the same channel) and to interact closely without having to physically hand a keyboard back and forth, or without having to walk back and forth between offices.

Another important SAIL facility is the *line editor* provided by the timesharing system. Input is normally read a line at a time by the user-level program, with each line being subject to complete editing throughout the line by means of the line editor—this is the default system interface to every program. The line editor thus provides the user with a standard input-line editing environment in any context. One especially useful line-editor command retrieves the previous line of text for further editing and reentry to the system, in order to correct a mistake or simply to issue the same command over again.

Line editing is aided by the addition of two shift-like keys to a standard ASCII keyboard. These two new keys, called *control* and *meta*, are used to set the eighth and ninth bits of input characters that would otherwise be normal 7-bit ASCII. This WAITS control key should not be confused with the ASCII control key, as these two provide two different functions.

Another convenient feature of the WAITS display support for terminals is what is called the *wholine*. This is a continually updated two-line text display at the top of the screen showing information about (1) the program the user is running and (2) the system status as a whole.

The user information on the wholine provides enough data to indicate who is logged in, what he is doing, and what percentage of the CPU he is getting when trying to run. The other part of the wholine gives system-wide information, showing, in various ways, how loaded the system is. The load average and time and day are very commonly checked items in this part of the wholine.

The system wholine status thus lets the user know what to expect in the way of system response right now, and the user wholine status indicates the actual response that the user is getting. The user is never left in the dark wondering if his program is taking much longer than he expected ("should I stop it?") or if it is just not getting run by a heavily loaded system. And this information is continually updated without the user intervention required for the control-T command in TOPS-10 or TOPS-20; furthermore, because control-T doesn't have its own output area, its repeated use can cause relevant information to scroll off the screen.

5. E, the Display Editor

E is a fast, display-oriented text editor that makes use of the system line editor for editing individual lines. **E** provides many conveniences, including: the display (naturally) of a screen-size piece of text from the file being edited; fast random access

to individual text pages of the file; commands to pick up selected text and move it within a file or from one file to another while it is displayed in context; fast screen updating; command/text macros; the ability to undo many commands (like text deletions, general changes to the current page, and positional changes); and the option of using multiple windows to have several files visible on the screen and/or open for editing.

The most commonly used **E** commands require only a single character to be typed, usually with the control key held down. Less common commands and more dangerous commands require typing a command keyword. Text that is to be input to the file is typed through the line editor, with no use of control or meta except to correct the line being typed.

E manages the display screen efficiently to reduce output. Whenever the screen text needs to be updated, **E** outputs only enough text to correct the changed parts. This might involve moving old text to new positions on the screen, as well as outputting brand new text not formerly displayed.

E normally uses almost all of the display screen to present text from the files being edited. However, two lines at the top are left for the wholine, and a three-line area is used at the bottom of the screen for the *page printer*, which is a simulated piece of paper onto which any typeout can go and which scrolls when it fills up. The page printer is used (1) for the echoing of commands typed by the user, (2) for any prompts necessary, and (3) for any additional information requested.

The main part of the display is used to present text from the files being edited. An arrow (or cursor) is used to indicate the user's position within the current file.

If the user has several files open for editing at once, each such file appears in a separate *window*. The various windows can be all simultaneously visible (with only part of the screen available to each), or all completely overlapped (to display a full screen of text in each, with only one of them visible at a time), or anything in between. That is, each window can have any size and position on the screen, overlapping partially, totally, or not at all with other windows. Switching to a window brings its text 'on top' of any others so that it is totally visible. Because window switching is very fast, and because updating of text on a Data Disc display is almost instantaneous, most people usually keep all of their windows the maximum size (completely overlapping) and rely on window switching to check information from different windows. However, when it is necessary to view simultaneously two or more windows, say to compare texts, that is easily arranged.

For rapid editing of any file, **E** uses random access to the file's individual pages. Each page can contain as many lines as the user wants. At any given time, **E** has one page of the file in core[3] and the user can wander around on that page, editing it as needed, without having to reference the disk. When the user moves to another page, **E** automatically updates in place the disk version of the page that has been in core and then reads in the new page of interest from the disk. To facilitate this random access to pages within the file, **E** maintains, at the beginning of the file, a directory page listing the record numbers where all the pages start. The directory page also contains the text from the first line of each page, and **E**'s search command is capable of searching this directory text (which is always kept in core) to find and go to the page that a given string (e.g., a label) occurs on. In fact, **E** can move from anywhere right to the label desired, having searched only the directory page plus the page that the directory indicated the label was on.

There are basically two modes in **E** in which commands or text can be typed; we'll call these *normal* mode and *line editor* mode. The difference between these two modes is very slight, as essentially the same commands or text can be typed in either mode. In fact, the user generally doesn't even notice when he is changing modes, because that happens automatically in carrying out commands given by the user. However, the difference between these modes is important for its effect, as it allows **E** to take advantage of a major aspect of the operating system, namely the line editor.

As mentioned earlier, the line editor is the part of the operating system which allows the user to edit a displayed input line. The user is not limited to making changes at the end of the line, but can move around anywhere in the line, adding, deleting or replacing characters or groups of characters. A cursor is displayed to indicate the current position of editing within the line. Line editor commands for moving the cursor or deleting or adding text are typed with the control or meta key to distinguish them from plain text being typed into the line, which is typed with neither control nor meta.

Because the line editor is a resident part of the operating system, it can respond very quickly to the user's typing, to keep the display of the line being edited always up to date. No user program is run during this line editing, so the system never needs to do any time-consuming swapping to provide line editor updating. Furthermore, because the line editor is the user interface to all programs and the monitor, not just to **E**, the user always has the same line editing commands and facilities at his disposal.

---

[3] Actually, the user can have any contiguous group of text pages in core at once. 'In core' means in **E**'s address space in memory, as opposed to on the disk. **WAITS** is not a demand paging system, so the job currently running is entirely in core. With the large amount of physical memory on the **SAIL** system, very little swapping is done.

He doesn't need to learn different line editing commands (or subsets of commands) for different programs, and no line editing code ever needs to be inserted in user programs.

In **E**'s normal mode, anything typed by the user goes directly to **E** to be processed as a command. The most important **E** commands are single characters typed with the control key to indicate that they are commands instead of text. Four of the most common commands (to move up or down by a line or a windowful) are single-character commands not requiring the control key. But almost any other single character typed without control or meta is interpreted as text to be put on the current line. When such a character is typed, **E** goes into line editor mode, and the user is now interacting with the line editor, not **E**, for editing that one line to the extent desired.

When the user finishes editing the line, he might type carriage return, which would cause the whole line to be given back to **E** as input, whereupon **E** would store away (in core) the edited version of the line. The command that ended editing of one line might or might not cause another line to be loaded into the line editor. If so, then the user remains in line editor mode in the new line; if not, then the user has returned to normal mode

But again, the modes are really important only to **E**, not generally to the user, and there are two particular reasons for this.

First, to get into line editor mode from normal mode the user just types the line editor command or plain text that he wants. The command or text he typed is then processed by the line editor as if the user had already been in the line editor with the cursor at the beginning of the line. Similarly any **E** command given from the line editor causes **E** to execute the command and select the new mode, whether normal or line editor, automatically. So the user never has to explicitly give a command to change between the modes.

Second, in line editor mode, the display of the line being edited, although maintained by the system instead of **E**, is positioned on the screen exactly where E was already displaying the text of that line. Thus the user continues to see the line's text presented in its usual context on the screen, whether or not he has the line in the line editor.

Another important facility of **E** is the *attach buffer*, which can be used for moving, copying, deleting or comparing lines of text. The attach buffer is so called because any text in it has been temporarily removed (or possibly copied) from a file page and now travels along with the user during movement within the files being edited. The distinguishing feature of the attach buffer is that although its text is technically not on the page being edited, it is displayed at the user's current position on that page. This lets the user see the attach buffer text in the context where it would be if the user were to put it down (de-attach it, so to speak) on the page at the current position. At the same time, the region of the display showing the attach buffer is marked specially (e.g., with bars in the left margin or with bold text) so that the user can tell which text is attached and which is on the page.

Like any good editor these days, **E** provides macros, which can be established as sequences of characters (commands and text) whose typing is to be simulated. A macro can call other macros or even itself. To facilitate 'programming' with macros, **E** has *readonly variables*, which give such values as the current line number and page number, the size of the page, etc. These readonly variables, along with conditional commands and 'normal' variables which the user can set or test, allow the user to write macros that can do different things depending on the state of the edit.

Although there is no general *undo* command in **E**, most commands can be undone in order to correct a user mistake or simply to let the user back up his thinking. For example, if changes have been made to a page without writing them out to the disk, the *cancel* command can be used to cancel all those changes, causing **E** to read the page back in from the disk. This is a very important command, as it allows the user to recover from major mistakes without having to worry about whether the recovery is complete or not (as he might have to do if just undeleting some individually changed lines). The user doesn't write out changes to a page until he is surè they are what he wants, although he can experiment with any jumbling he wants of the text in core without any risk to the file.

Other conveniences in **E** include commands to: (1) send messages to other people's mail files or terminals, with the message text being typed in and/or coming from the file being edited; (2) justify paragraphs or other pieces of text; (3) find matching parentheses (or other delimiters); (4) specify the range of a subsequent command to be the current paragraph or message, without having explicitly to find where that range starts or ends; (5) spool selected pieces of text to available printers; (6) rename and delete files; and (7) save in an emergency file the edited text that is in core at the moment of any disaster, whether a rare **E** bug, a memory parity error, or a user mistake.

6. The Interface

The basic idea behind our interface is that each user should be able to use the system-provided editor as the basis of interaction with every program. On SAIL, a high-speed display terminal and **E** can be used to manipulate text as the input to programs and to manage the output text onto the screen and/or into a file. This is accomplished by maintaining one or more jobs controlled by **E** using a message-passing protocol.

In this way the user can operate a subsystem such as Lisp from within **E** while observing the source code for that program or subsystem. This is useful for single-stepping through the code, for examining program state while at a breakpoint, for modifying the source and executing code after bugs or deficiencies have been identified, and for making transcripts of interactions.

The user can maintain a file of working text that contains commonly-used expressions for testing code, obtaining the current program state, and giving commands to the subsystem. By pointing to appropriate pre-constructed text expressions within this file, the user can send the text as input to the subsystem, thus eliminating the necessity to re-type common commands and avoiding errors in such retyping. This allows the user to remember the debugging state more easily.

A task many users want to accomplish is to get program output into a file. With Lisp this requires opening an output file and writing to it. When the program is designed to have terminal IO only, getting this output to a file may not be so easy or convenient. With this interface it is trivial, because the 'terminal' is an editor window into a file.

E can accept commands from a subsystem exactly as it does from the user, so that the subsystem can be used as a 'macro' language for the editor. We will talk more about this later.

Because **E** supports multiple windows, any subsystem operating under this interface can easily utilize multiple windows within the editor.

One of our premises is that a *text* editor rather than a *structure* editor is appropriate, even for languages like Lisp in which balancing of delimiters—parentheses—is crucial. Editing a program is the same physical act as editing text, and one faces the least amount of mental effort when the tools are the same for both program and text editing.

However, an editor that understands the language being edited *is* useful. **E** understands the Lisp language to a certain degree, and the **E** interface to Lisp provides us with the means to introduce further knowledge into the environment.

For brevity we will discuss our interface using the Lisp subsystem as our main example, though there is nothing special about Lisp to our interface. The interface can work as well with any other subsystem.

Actually, **E** can support any number of subjobs this way. In fact, with the pseudo-terminal (PTY) feature of WAITS, any program or subsystem can be run under this interface with no modification to the program or subsystem.

6.1 *The E-Lisp Interface*

Now we will describe some of the specific operations that the user can perform with the E-Lisp interface. Generally, the user will be editing one or more files, including some sources and working text, while debugging a Lisp program through the E-Lisp interface.

In **E** it is possible to send any number of lines of text to Lisp to be evaluated. When Lisp sends results back to **E**, the text it returns can be: viewed at the bottom of the screen, inserted at the end of the current file page, inserted before the current line, or placed at the end of the attach buffer. With these modes it is easy to obtain programming-session transcripts.

As an example of the use of these features, it is simple to move the cursor to some program code and have Lisp expand all macro calls appearing in that code, so that the programmer can see the actual Lisp code that would be executed. Lisp programmers write many macros, and the common, difficult-to-debug problem of incorrect macro definitions is attackable this way.

One can go into a special *EVAL* mode in **E**, which allows the user to type text directly to Lisp. While one is in *EVAL* mode, output from Lisp will go to the selected destination, for example, into the file being edited.

Lisp is able to send commands to **E**, which will be executed exactly as if those commands were issued by the user; thus the Lisp program is able to 'edit' the files it is being controlled through. To aid in this, it is possible for Lisp to obtain the values of **E**'s readonly variables.

A final, useful feature is that the Lisp reader is able to issue **E** commands while reading an S-expression. Recall that Lisp's top level operation is to read an S-expression, to evaluate that expression, and finally to print its value. Within the interface this is equivalent to receiving a message from **E**, constructing an S-expression from the message text, evaluating that S-expression, and sending back a message to **E** containing the result. In this context one can set up a Lisp 'demon' which will be invoked whenever the current message from **E** is emptied before an S-expression is complete. This demon can then invoke **E** commands to send the remainder of the S-expression. We call this the 'buffer-dry' demon.

An important point is that **E** and Lisp are separate jobs running in a timeshared environment, so the user can edit while his code runs along in parallel.

6.2 *An Example*

Suppose we have the Fibonacci program:

```
(defun fib (n)
       (cond ((= n 0) 0)
             ((= n 1) 1)
             (t (break fib t)
                (+ (fib (- n 1))
                   (fib (- n 2))))))
```

in a file. We get into **E**, editing this file, place the arrow so that it points to any line of this definition, and type $\alpha!\alpha=$.[4] This causes **E** to send the entire function definition to Lisp: The $\alpha!$ tells **E** that the next command applies to the entire current paragraph—in this case the definition of **fib**; the $\alpha=$ is the command that causes **E** to send text to Lisp. In this case it causes the entire function definition to be sent to Lisp. We type $\alpha 0\alpha$=(fib 20) to evaluate (fib 20) immediately. $\alpha 0\alpha=$ causes **E** to send the line of text about to be typed to Lisp. Lisp hits the breakpoint, and we can examine the current environment. Now we make sure that the page containing this definition has been written out to disk—we intend to alter the page to do some experiments.

First we change the definition by removing the breakpoint, and then we type $\alpha!\alpha=$ to re-define **fib**. Now we move down to the next to last line and edit it to be: (**setq** x (fib (− n 1))). With the arrow at this line we type $\alpha=$. This sets the value of x to be (fib 19) because 20 is the value of n; in addition, 4181 is printed in the page printer, because the value of the **setq** is the value of (fib 19). We can check the value of n by typing $\alpha 0\alpha=$ n, which would evaluate n in the current environment.

Now we type α–αX LRECEIVE, which means **E** will not receive any values from Lisp until the user types αX LRECEIVE; and we type αX LFILE, which means **E** will put future text returned by Lisp just above the arrow line. We place the arrow at the last line in the definition and type $\alpha=$ to evaluate (fib 18). Eventually **E** signals that the result has arrived. We place the arrow at a line below the function definition and type αX LRECEIVE. **E** places 2584 on a new line above the arrow.

We type αX LTYPE to have further output from Lisp go to the page printer, edit the line with 2584 on it to read (+ x 2584), place the arrow at that line, and type $\alpha=$ to get the value of (fib 20), which is 6765. After checking these values in Knuth, Volume 1, we type αX CANCEL to undo changes to this page, and our debugging session is over.

6.3 *Another Example*

Here is a function written in Lisp that will cause **E** to send the current text item—the item that would be sent if the user were to type $\alpha=$:

```
(defun send-current ()
       (em:ecommands '(α =))
       (read -em:sfa-))
```

em:ecommands takes a list of characters; it makes up a message containing those characters as 'E-commands' and sends it to **E**. Then **E** runs the contents of that message exactly as if it were a macro. Here the message is $\alpha=$, which will cause **E** to send to Lisp either the current line or the contents of its attach buffer, if any (this is exactly what $\alpha=$ does). Next **em:ecommands** does a **read** from -em:sfa-, which is bound to the stream that **E** supplies. Thus, this piece of code returns the S-expression that it instructs **E** to send to it.

6.4 *Implementation*

This section describes in some detail the implementation of the **E**-Lisp interface. These details are presented to sustain the claim that little additional operating system functionality is needed to interface any system-supplied text editor to a Lisp subsystem.

Because **E** and Lisp run as two separate jobs, messages are passed between them to communicate. There are two ways of doing this at the system level—one handles small messages rapidly, and the other handles larger messages more slowly.

WAITS allows one job to start another as a detached job (a job not connected to a terminal). Lisp is started up as a detached job in such a way that it knows to interact with the initiating **E** job.

Message-passing is done with the MAIL UUO [Frost 1977] (an operating system call), which enables one job to send a small number of words of data to another job via a system buffer.[5] If the data part of a message will fit within the MAIL UUO's data block, then it is passed by the MAIL UUO itself; otherwise a pointer to the data within the sender's job is passed, and the receiver uses another UUO—the JOBRD (job read) UUO—to

[4] As a notational expedient, the use of the control and meta keys is indicated by using the Greek characters 'α' for control and 'β' for meta, with these placed before the character of interest. For example, αA is read as 'control-A,' βB is 'meta-B,' and $\alpha\beta$C is 'control-meta-C.' Remember that the control and meta keys are shift-like keys; thus $\alpha\beta$C means 'hold down the control and meta keys while typing C.'

[5] The MAIL UUO is merely for passing a fixed amount of data between two jobs; because of the name of the UUO, this data is sometimes called a 'letter.' However, this UUO has nothing to do with electronic mail systems that allow memos to be sent between users.

copy the given block from the sender's job. Because the JOBRD UUO has a maximum transfer size (1K words), when the message is larger than this, it is broken into a chain of blocks small enough for JOBRD.[6]

6.4.1 System Message Primitive

Some of the features of the MAIL UUO that we use to gain the functionality necessary for the interface are:

1. Sending a letter to a particular job, signalling an error if there is already a letter in its mailbox. This is used to test whether there is unread mail for the other job and to send some, if not, in one step. Leaving mail unread is one of the few ways to synchronize the two processes.
2. Waiting for mail to be sent from the other job. This is useful when one job runs ahead of the other.
3. Reading a letter if there is one, and signalling an error if there isn't a letter to be read. This is mainly used in cases when it is known already that there is a letter in the mailbox.
4. Receiving an interrupt when a letter arrives. This feature is necessary for the interface to work smoothly, yet it is a source of programming anguish because a fair amount of work has to be done when an interrupt occurs.

6.4.2 Basic Messages

The basic message that is sent is a structure with four fields: a validation, a message descriptor, a data descriptor, and data. The validation is to ensure that the mail is from the correct job and that it is a message following the defined protocol. The message descriptor tells what type of message this is. The options are: (1) short text, (2) long text, (3) long-text acknowledgment, (4) long text to be continued (sent with the first part of the long text), (5) **E** commands (from Lisp), (6) explicit end of file (from **E**), (7) request for readonly variables (from Lisp), (8) values of readonly variables (from **E**), (9) interrupt character (from **E**), and (10) kill-self command (from **E**). The data descriptor tells how many bytes are being sent and (in long messages) the location of the data in the sender's address space. The data field contains the data itself if it fits; for long text transfers, in which the data is transferred separately by the receiver, the data field of the MAIL UUO is unused. If the message descriptor indicates an 'interrupt character' (number 9 above), the character is in the data descriptor field.

In the case of a short text transfer, because the data is in the message itself, there is no need for the sender to wait for an acknowledgment of receipt. If a second message follows on the heels of this one, and if the receiver has not yet processed the mail, the second MAIL UUO will fail to deliver the message and will report that the receiver's mailbox is full. Thus the sender will wait on the receiver. In the long-text case, because a large buffer in the sender's address space must be used for the message, the protocol requires an acknowledgment from the receiver once the receiver has transferred the data with the JOBRD UUO; the sender must not re-use the buffer until the acknowledgment has been received. Significant waiting can occur before a message transfer is completed because both **E** and Lisp dynamically allocate storage for buffers, and, in Lisp's case, a garbage collection may be needed to gain space for the buffers.[7]

6.4.3 Message Details

Text is sent in 9-bit bytes from **E** to Lisp (because Lisp uses 9-bit input, including control and meta keys, in normal, direct usage) and in 7-bit bytes from Lisp to **E**. Commands from Lisp to **E** are in the so-called **E** macro format, which uses 7-bit bytes to represent 9-bit command text. For example, control-P, the **E** command to move to the next page, is passed as 'αP' ('α'[8] followed by 'P'), and an escape character is available for passing characters like a real 'α.'

Readonly variable requests are in SIXBIT,[9] with variable-value pairs returned by **E**.

As noted, the Lisp reader takes 9-bit characters. This is due partly to the fact that control characters are used for user interrupts by Lisp. For example, if a computation is taking longer than anticipated, the user may suspect some problem and will want to interrupt Lisp in order to examine the execution state. He would normally type 'αB' directly to Lisp, which would signal a breakpoint interrupt to Lisp. Now, if the **E**-Lisp interface didn't allow for interrupts, the user would have to either wait out the Lisp job or kill it in this suspicious runaway case. Therefore, a message type of 'interrupt' tells Lisp to immediately decode the message, processing the interrupt character just as always, so that the user can interrupt a computation and resume it exactly as he would be able to with a raw Lisp.

---

[6] We want to allow programs to need only a single 1K buffer for simple interactions, so the protocol supports this chaining mode.

[7] Actually, because text requiring several long messages is very rare, and because garbage collection is a slowdown, a 1K buffer (the maximum size for the JOBRD UUO) is permanently allocated for long transfers, and subsequent buffers are allocated dynamically).

[8] In **WAITS** ASCII, 'α' is a printing character; as mentioned earlier, it is commonly used to denote use of the control key with the following character.

[9] SIXBIT is a compact representation for a subset of ASCII characters.

In addition to handling user-requested interrupts, Lisp must be able to accept messages at any time in order to achieve a necessary degree of user convenience. Let us demonstrate why. Suppose that Lisp were *not* able to receive and queue messages at arbitrary points in the interaction. That is, suppose that Lisp would always process a message, coming to a quiescent state, before handling any further message traffic. 'Processing' a message means constructing an S-expression from the message text and then evaluating that S-expression. This can be shown inadequate if you suppose the user sends a message to Lisp requesting a long computation and follows that message with another request immediately, wanting then to edit for a while, and if that followup is a long message requiring an acknowledgment, then if Lisp were to process its first message before acknowledging the second, the user would not be able to edit while **E** awaited Lisp's response.

Similarly, if several short messages are sent in a burst, then the interface would balk because the sender would find the receiver's mailbox full a lot of the time. These situations would imply a synchronous and relatively ineffective interface because **E**, and hence the user, would spend a lot of time waiting for Lisp to accept messages.

Therefore, Lisp has been made capable of accepting, queuing, and acknowledging messages from **E** at virtually any time. It does this after getting a MAIL UUO interrupt by suspending the current computation, executing the message protocol to read and queue the message, and then returning to the suspended computation.

E is also able to queue mail, but not as flexibly as Lisp. It happens that all interactions can be efficiently handled if only one of the processes can queue arbitrarily.

The buffer-dry demon, mentioned earlier, is implemented quite straightforwardly: whenever the Lisp reader needs to wait for **E** input, a global variable is probed to see whether it is NIL or has some value. If it has a value it is assumed to be a user function of no arguments. The state of the computation is saved, and this function is applied. Normally this function will cause **E** to send some more text, possibly after probing the files **E** is using to find the right text to send.

6.5 *Some Niceties*

Lisp has various internal options to make the **E**-Lisp interface comfortable for the user. Lisp can send back its response text after every line (the default), after every n lines, or never. And there are primitives to force unsent buffers to **E**, to treat messages as files,[10] to position **E**'s cursor at various points meaningful to Lisp (at DEFUNs for example), to go into any of several transcript modes, to balance parentheses given an annotated expression identifying matching points, to cause the macro expansion of the text within **E**'s attach buffer, and to map a function over a file of text line by line. Many of these primitives require the two-way nature of the interface.

E is capable of waiting for exactly or at least n lines to be sent from Lisp before proceeding, which is useful for writing **E**-Lisp macros.

Lisp programs and systems need not know that they are running within the interface. Adapting any Lisp-based subsystem to the interface takes only a reload of the subsystem along with the interface code.

[10] This allows one to write loops nearly identical to those which process disk files. End of message is taken as end of file.

7. Conclusions

The main points of this paper are:

1. the standard system editor can be the interface for all programs;
2. text editors are at a good level for uniform interfaces;
3. timeshared computers can provide advanced programming environments; and
4. only simple timesharing system and hardware features are needed to implement such an environment.

We have demonstrated that programming environments in timesharing systems can be made more effective by providing a common editor interface to the entire system.

8. References

[**Frost 1977**] Frost, Martin; **UUO Manual**, Stanford Artificial Intelligence Laboratory Operating Note 55.5, 1977.

[**Greenblatt 1983**] Greenblatt, Richard; Personal communication, Lisp Machine Inc., 1983.

[**Harvey 1983**] Harvey, Brian; Frost, Martin; **Monitor Command Manual**, Stanford Artificial Intelligence Laboratory Operating Note 54.7, 1982.

[**Moon 1983**] Moon, David; Daniel Weinreb; **Lisp Machine Manual**, Symbolics, Inc., 1983.